

Team Presentation

Team 5

Qingdao University of Technology

October 5, 2022

1 Problem 1

2 Problem 2

3 Problem 3

4 Problem 4

5 Problem 5

Problem 1

Design two nonlinear algebraic equations (degree at least 5 , with no less than 3 zeros) and find all their zeros (error less than 0.001),show your result and Matlab or Python code.

Problem 1 Solution

First, draw the graph of the higher degree equation of one variable then find the approximate interval between zero, Finally, the binary method is used to determine the value of zero.

Following are our target functions:

$$y = x^6 + x^4 - 3x^5 - 2x^3 + 4x^2 \quad (1)$$

$$y = x^7 - 2x^6 - 4x^5 + 3x^3 + 3x^2 \quad (2)$$

Problem 1 Matlab Code

```
1 function r=bisect(a,b,accurate)
2 p=(a+b)/2;
3 error=b-a;n=0;
4 while (error>accurate)
5     n=n+1;
6     if fun(p)==0
7         break;
8     else if fun(a)*fun(p)<0
9         b=p;
10    else
11        a=p;
12    end
13    p=(a+b)/2;
14    error=(b-a)/2^2;
15 end
16 r=p;
17 end end
```

Problem 1 Solution

By using bisection algorithm, we got the approximation of the function's root:

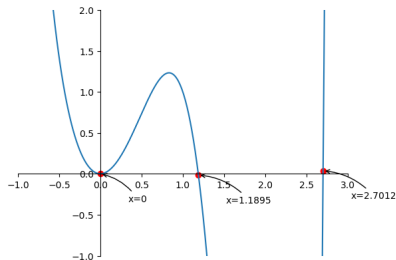


Figure: $y = x^6 + x^4 - 3x^5 - 2x^3 + 4x^2$

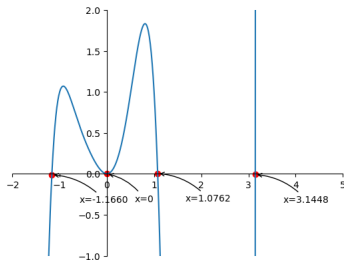


Figure: $y = x^7 - 2x^6 - 4x^5 + 3x^3 + 3x^2$

Problem 2(1)

Find the numerical solution of the following nonlinear equations using iteration method and Newton method respectively and show your analysis of error by using norms of vector and matrix and demonstrate your computer code

$$\begin{cases} x_1^2 - 10x_1 + x_2^2 + 8 = 0 \\ x_1x_2^2 + x_1 - 12x_2 + 7 = 0 \end{cases} \quad (3)$$

Problem 2(1) Solution

We put

$$F(X) = \begin{pmatrix} x_1^2 - 10x_1 + x_2^2 + 8 \\ x_1x_2^2 + x_1 - 12x_2 + 7 \end{pmatrix} \quad (4)$$

,and

$$X = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (5)$$

And the Jacobian matrix

$$J(X) = \begin{pmatrix} 2x_1 - 10 & 2x_2 \\ 1 + x_2^2 & 2x_1x_2 - 12 \end{pmatrix} \quad (6)$$

Initial guess $X^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$.

Problem 2(1) Solution

Thus

$$F(X^{(0)}) = \begin{pmatrix} 8 \\ 7 \end{pmatrix} \quad (7)$$

$$J(X^{(0)}) = \begin{pmatrix} -10 & 0 \\ 1 & -12 \end{pmatrix} \quad (8)$$

By using

$$J(X^{(n)}) \Delta X^{(n)} = -F(X^{(n)}) \quad (9)$$

We got $\begin{pmatrix} -10 & 0 \\ 1 & -12 \end{pmatrix} \Delta X^{(0)} = \begin{pmatrix} -8 \\ -7 \end{pmatrix}$, and $\Delta X^{(0)} = \begin{pmatrix} 0.85833 \\ 0.58333 \end{pmatrix}$.

Problem 2(1) Solution

By using $\Delta X^{(k)} = X^{(k+1)} - X^{(k)}$,
we had

$$X^{(1)} = \Delta X^{(0)} + X^{(0)} = \begin{pmatrix} 0.85833 \\ 0.58333 \end{pmatrix} \quad (10)$$

The rest can be done in the same manner. Following are each iteration result.

	1	2	3	4
x_1	0.85833	0.93589	0.93667	0.93668
x_2	0.58333	0.69959	0.69955	0.69959

Problem 2(1) Solution

By using norms of vector and matrix, we analysis the error

$$\|X^n - X\| \leq \left\| \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{pmatrix} \right\| \wedge n \|X^{(0)} - X^{(1)}\| \leq 10^{-5} \quad (11)$$

Problem 2(1) Fixed-Point Python Code

```
1 x = np.zeros([max_iteration, 2], dtype='float64')
2 err = np.zeros(max_iteration - 1, dtype='float64')
3 distance = 1; idx = 1; x[0, 0] = 1; x[0, 1] = 1
4 while(distance > acc):
5     if(idx < max_iteration):
6         x[idx, 0] = ((x[idx - 1, 0])**2 + (x[idx - 1, 1])**2 + 8) / 10
7         x[idx, 1] = (x[idx - 1, 0] + (x[idx - 1, 0] * x[idx - 1, 1])**2 + 7) / 12
8         distance = (x[idx - 1, 0] - x[idx, 0])**2 + (x[idx - 1, 1] - x[idx, 1])**2
9         err[idx - 1] = distance
10        idx = idx + 1
11    else:
12        break
```

Problem 2(1)

In a very extreme condition like in a tolerance of 1×10^{-20} , the fixed-point iteration took about 20 times to reach the target.

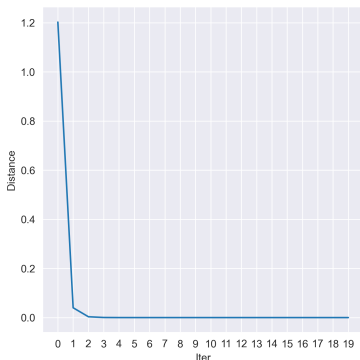


Figure: Fixed-Point Iteration Result

Problem 2(1) Newton-Raphson Matlab Code

```
1 syms x1 x2;
2 f1(x1,x2) = x1^2 - 10*x1 + x2^2 + 8;
3 f2(x1,x2) = x1*x2^2 + x1 - 12*x2 + 7;
4 F(x1,x2) = [f1;f2]; x_solve(:,1) = [1;1];
5 J(x1,x2) = jacobian(F,[x1 x2]);
6 dx_Nr=[0;0];
7 for i = 1:N
8     dx_Nr(:,i) = J(x_solve(1,i),x_solve(2,i))\(-F(x_solve(1,i),x_solve(2,i)));
9     if(sqrt(sum(dx_Nr(:,i).^2)) < epsilon)
10         x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
11         break;
12     else
13         x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
14     end end
```

Problem 1 Solution

Using programming tools, we got the root of this given nonlinear system. In the tolerance of 1×10^{-10} , the two methods showed a very different performance. Newton-Raphson iteration converges in incredible speed which only took 4 times to get a trustable root.

$$\begin{cases} x_1 = 0.936680033263737 \\ x_2 = 0.699593344681334 \end{cases} \quad (12)$$

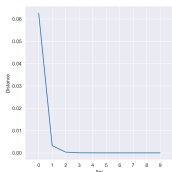


Figure: Fixed-Point Iteration

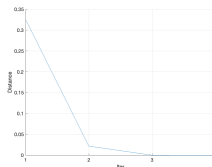


Figure: Newton-Raphson Iteration

Problem 1 Solution

Using programming tools, we got the root of this given nonlinear system. In the tolerance of 1×10^{-10} , the two methods showed a very different performance. Newton-Raphson iteration converge in incredible speed which only took 4 times to get a trustable root.

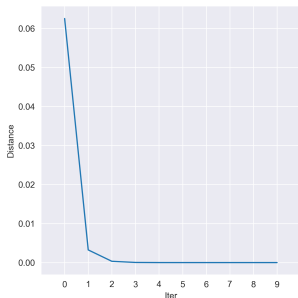


Figure: Fixed-Point Iteration

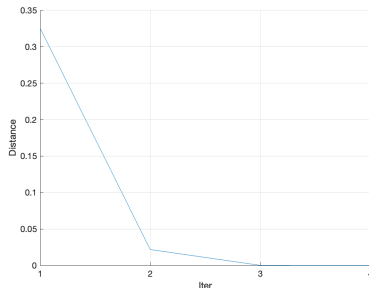


Figure: Newton-Raphson Iteration

Problem 2(2)

Find the numerical solution of the following nonlinear equations using iteration method and Newton method respectively and show your analysis of error by using norms of vector and matrix and demonstrate your computer code

$$\begin{cases} 5x_1 - x_2^2 + x_1x_3 = 2 \\ x_1x_3 + 4x_2 - x_3 = 0 \\ x_1^2 + x_2 - 3x_3 = 1 \end{cases} \quad (13)$$

Problem 2(2) Solution

We let $X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$ and $F(X) = \begin{pmatrix} 5x_1 - x_2^2 + x_1x_3 - 2 \\ x_1x_3 + 4x_2 - x_3 \\ x_1^2 + x_2 - 3x_3 - 1 \end{pmatrix}$. So the Jacobian matrix of $F(X)$ equals to

$$\begin{pmatrix} 5 + x_3 & -2x_2 & x_1 \\ x_3 & 4 & -1 \\ 2x_1 & 1 & -3 \end{pmatrix} \quad (14)$$

Problem 2(2) Solution

At first we guess $X^{(0)} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$,

so we got $F(X^{(0)}) = \begin{pmatrix} 3 \\ 0 \\ 0 \end{pmatrix}$, $J(X^{(0)}) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 4 & -1 \\ 2 & 1 & -3 \end{pmatrix}$.

By using

$$J(X^{(n)}) \Delta X^{(n)} = -F(X^{(n)}) \quad (15)$$

we have

$$\begin{pmatrix} 0 & 0 & 1 \\ 0 & 4 & -1 \\ 2 & 1 & -3 \end{pmatrix} \Delta X^{(0)} = \begin{pmatrix} 3 \\ 0 \\ 0 \end{pmatrix} \quad (16)$$

Thus, the solution is $\Delta X^{(0)} = \begin{pmatrix} -0.439412 \\ 0 \\ -0.352941 \end{pmatrix}$.

Problem 2(2) Solution

Next, by using $\Delta X^{(k)} = X^{(k+1)} - X^{(k)}$,
we have

$$X^{(1)} = \Delta X^{(0)} + X^{(0)} = \begin{pmatrix} 0.470588 \\ 0 \\ -0.352941 \end{pmatrix} \quad (17)$$

The rest can be done in the same manner. Following are each iteration result.

	1	2	3	4
x_1	0.470588	0.423831	0.424716	0.4247
x_2	0	-0.042279	-0.041371	-0.0413
x_3	-0.352941	-0.288278	-0.286962	-0.2870

Problem 2(2) Solution

By using norms of vector and matrix, here exists

$$\left\| X^{(n)} - X^{(0)} \right\| \leq \left\| \begin{array}{ccc} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} \end{array} \right\| \wedge n \left\| X^{(1)} - X^{(0)} \right\|, \quad (18)$$

and we get

$$\left\| X^{(4)} - X^{(0)} \right\| \leq \left\| \begin{array}{ccc} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} \end{array} \right\| \wedge 4 \left\| X^{(1)} - X^{(0)} \right\| \leq 10^{-5} \quad (19)$$

Problem 3(1)

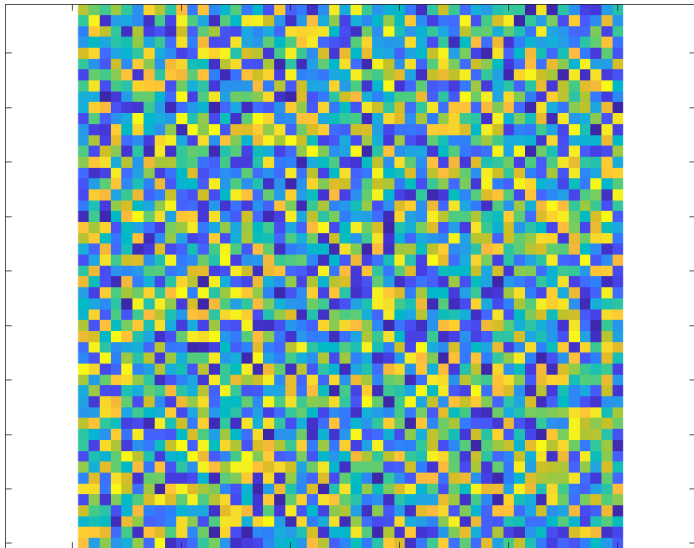
Generate matrix $A(50 \times 50)$ and vector $b(50 \times 1)$ by using random number and solve the linear system $AX = b$ by using Gaussian partial pivoting elimination algorithm.

```
1 clc;clear all;
2 n=50;
3 A=randi(100,n); %生成n阶方阵，元素取值范围为0-100
4 b=randi(100,n,1); %生成n*1向量，元素取值范围为1-100
```

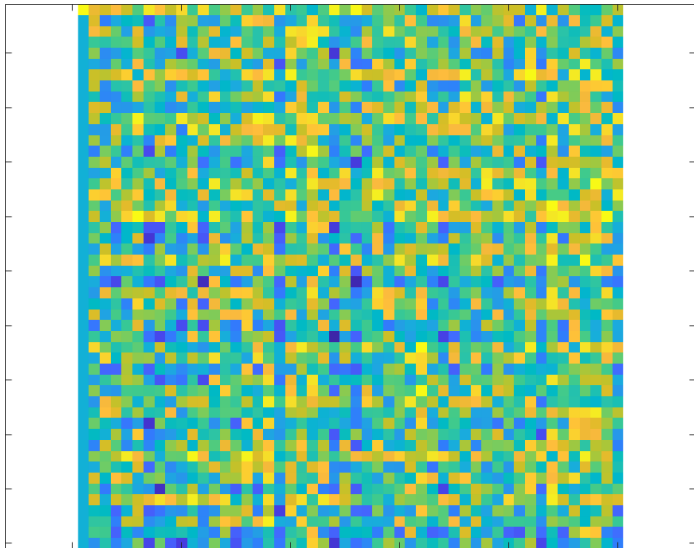
Problem 3(1) Solution

```
1 for i=1:n
2     for j=i+1:n %选取绝对值最大的为列主元
3         if(abs(T(j,i))>abs(T(i,i)))
4             max=T(j,i);
5             Temp=T(i,i);
6             T(i,i)=T(j,i);
7             T(j,i)=Temp;
8         end
9     end
10    for k=i+1:n %计算得上三角矩阵
11        m_ik=T(k,i)/T(i,i);
12        T(k,i+1:n+1)=T(k,i+1:n+1)-
13        (T(i,i+1:n+1).*m_ik);
14        T(k,i)=0;
15    end
16    imagesc(T(:,1:n)); %画图
17    axis equal
18    F=getframe;
19    imwrite(F.data, strcat('./
img/',num2str(i),'.png'));
20 end
```

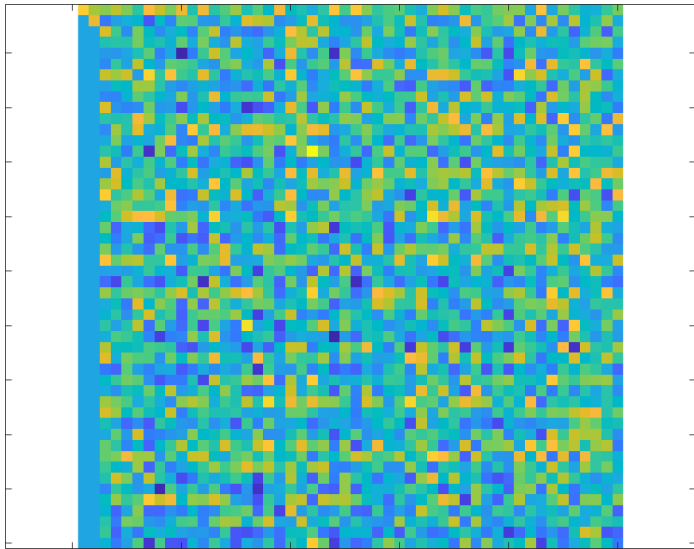
Problem 3(1) Solution



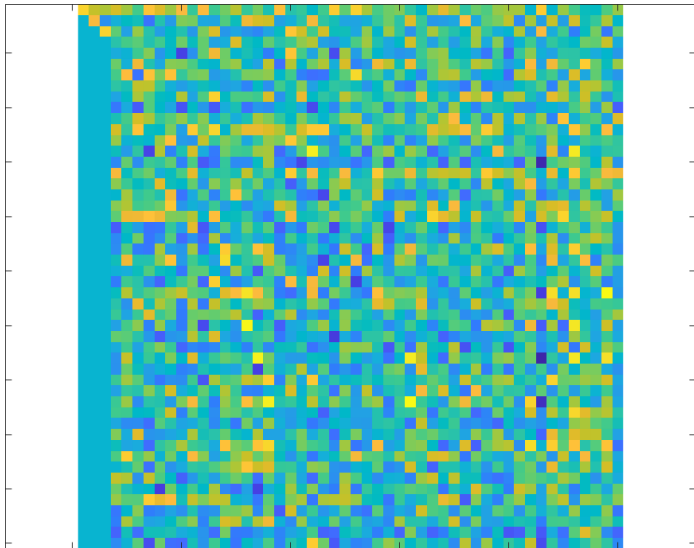
Problem 3(1) Solution



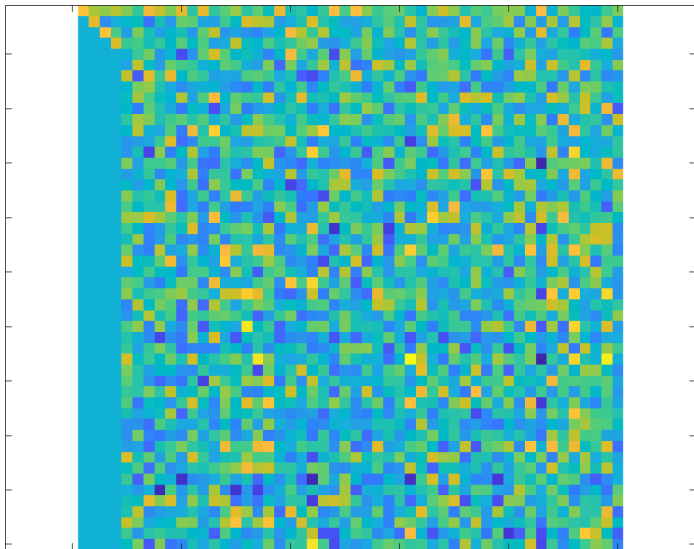
Problem 3(1) Solution



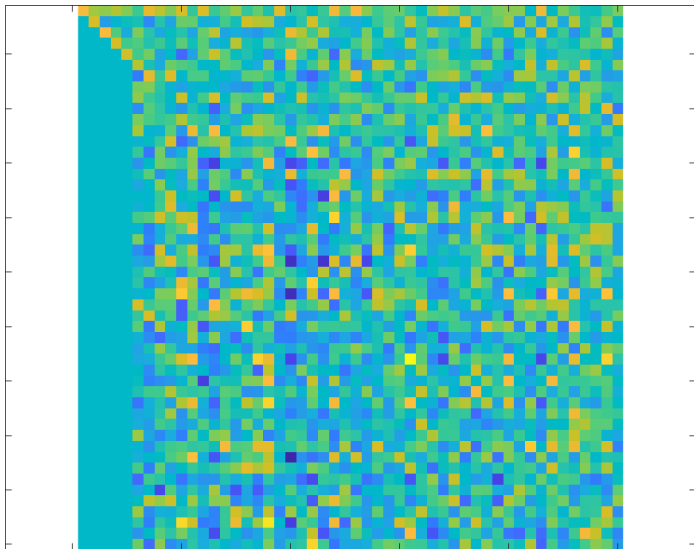
Problem 3(1) Solution



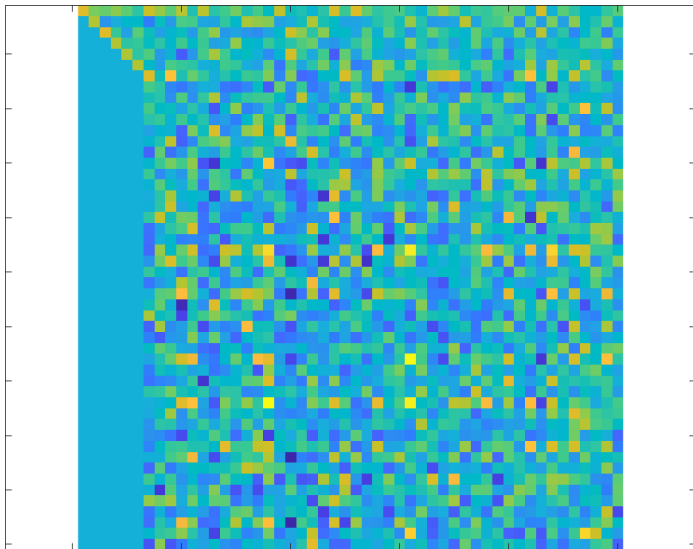
Problem 3(1) Solution



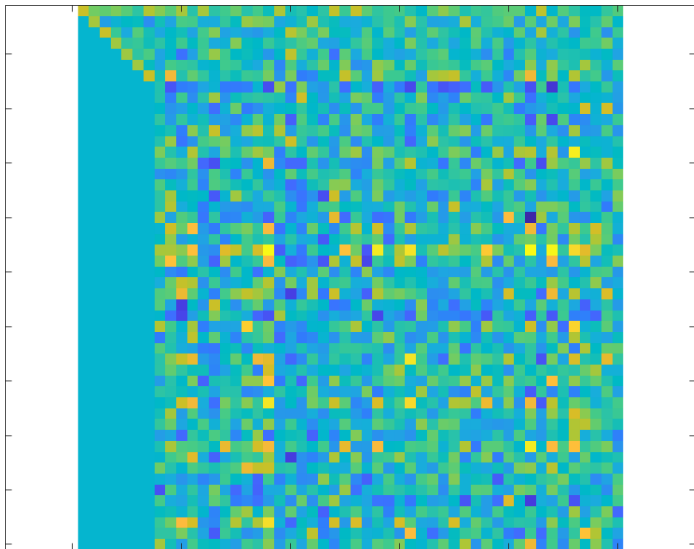
Problem 3(1) Solution



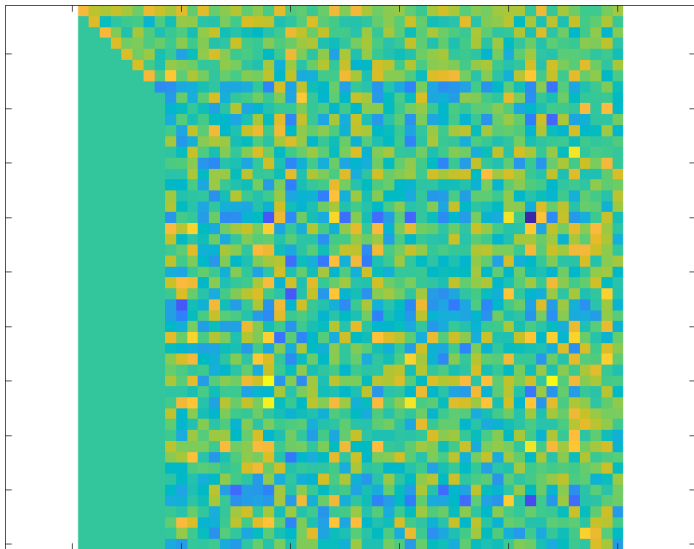
Problem 3(1) Solution



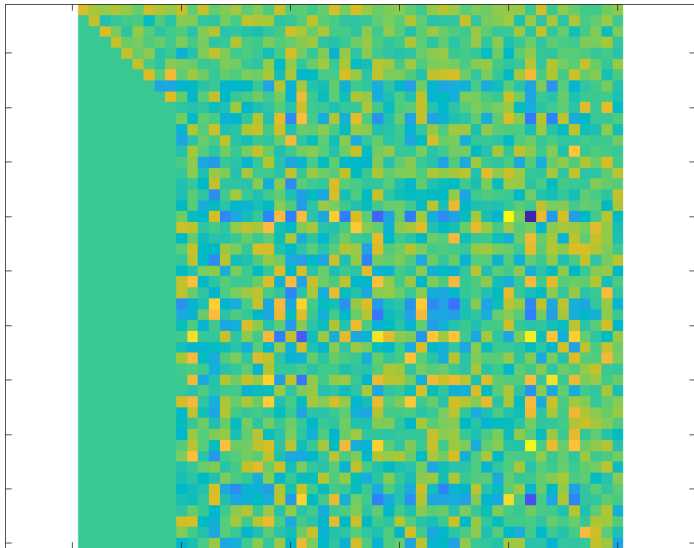
Problem 3(1) Solution



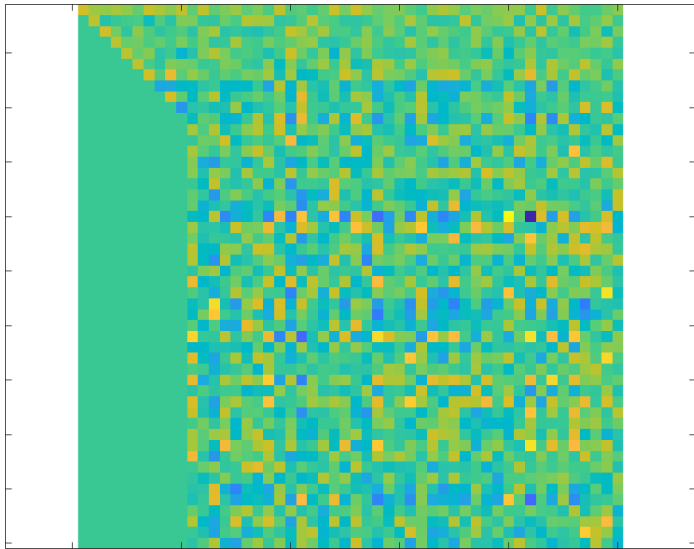
Problem 3(1) Solution



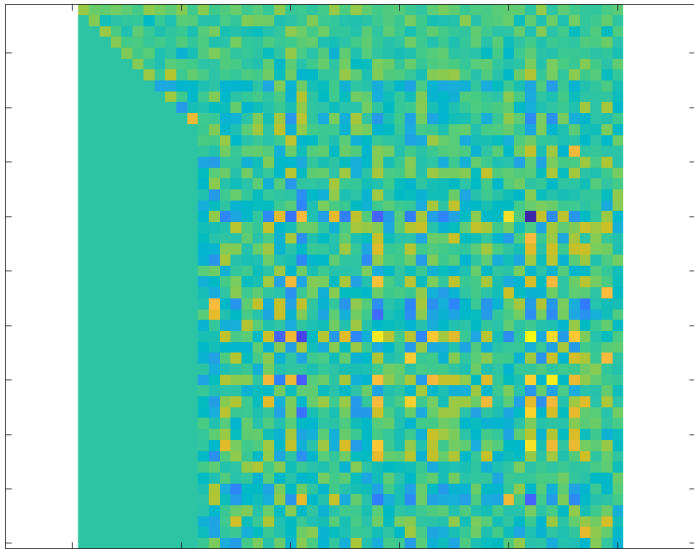
Problem 3(1) Solution



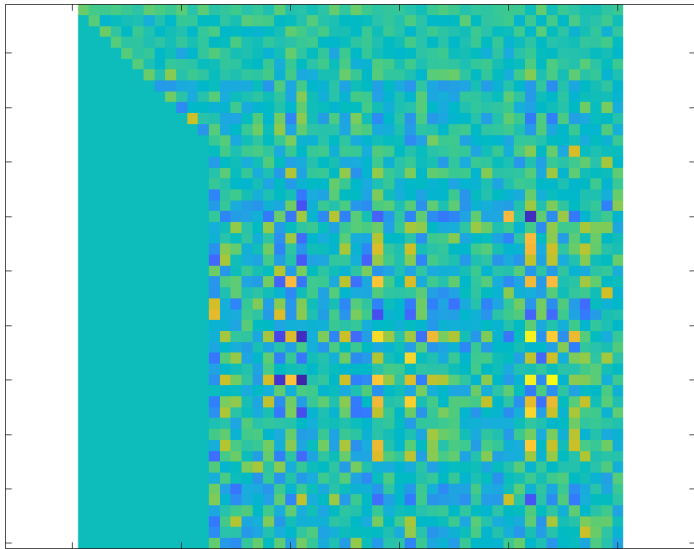
Problem 3(1) Solution



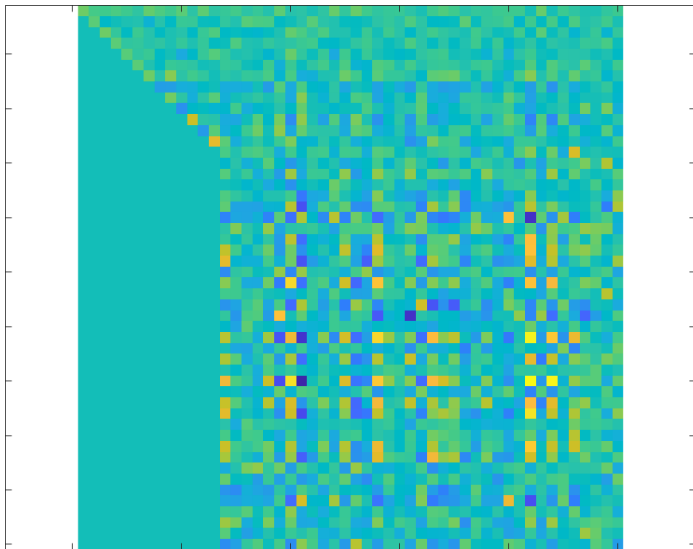
Problem 3(1) Solution



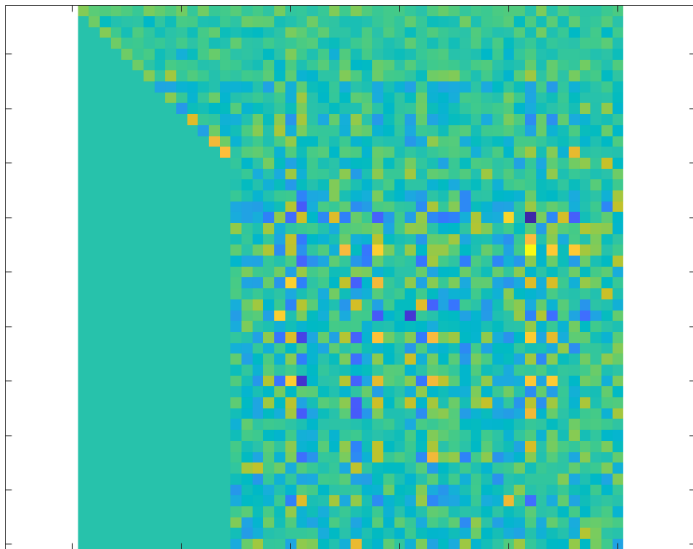
Problem 3(1) Solution



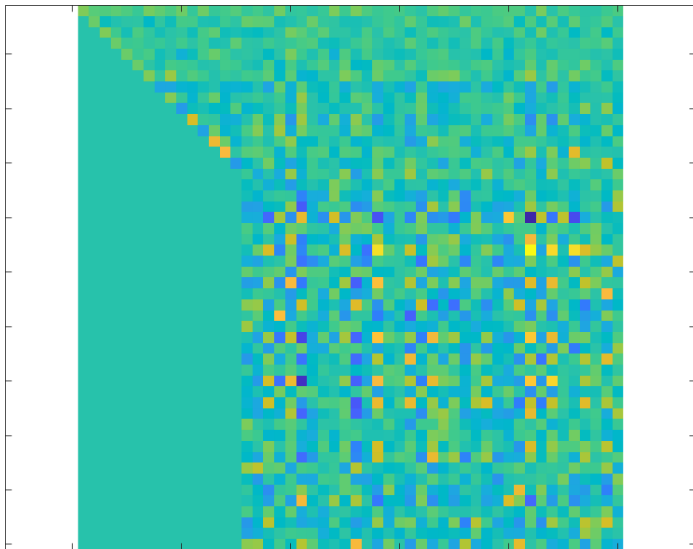
Problem 3(1) Solution



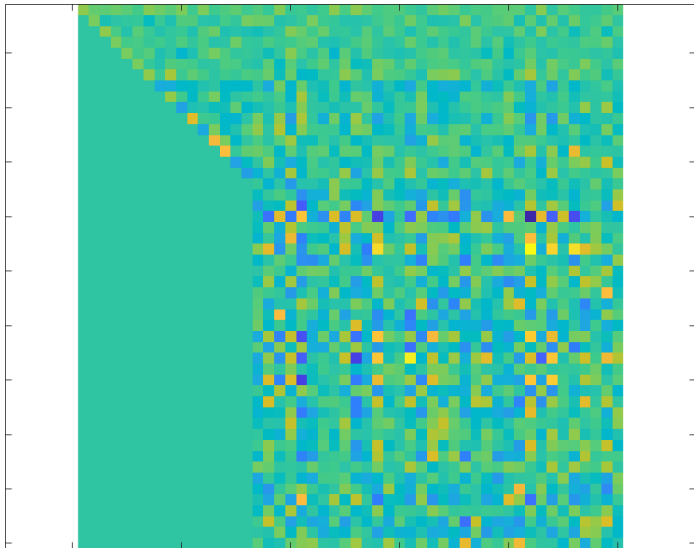
Problem 3(1) Solution



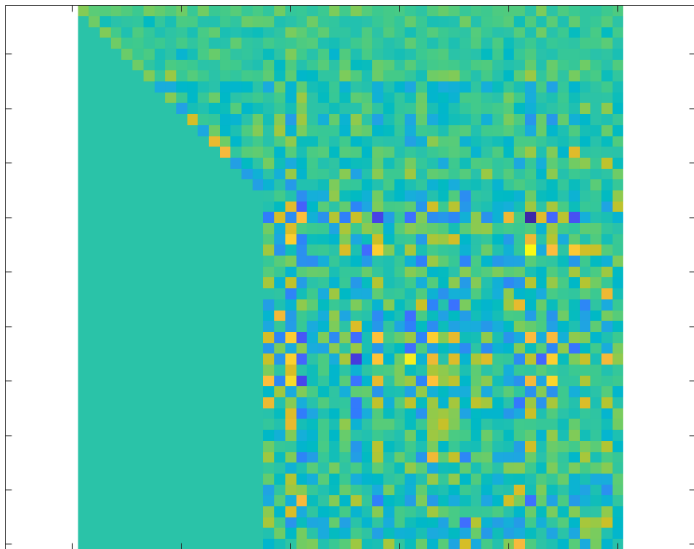
Problem 3(1) Solution



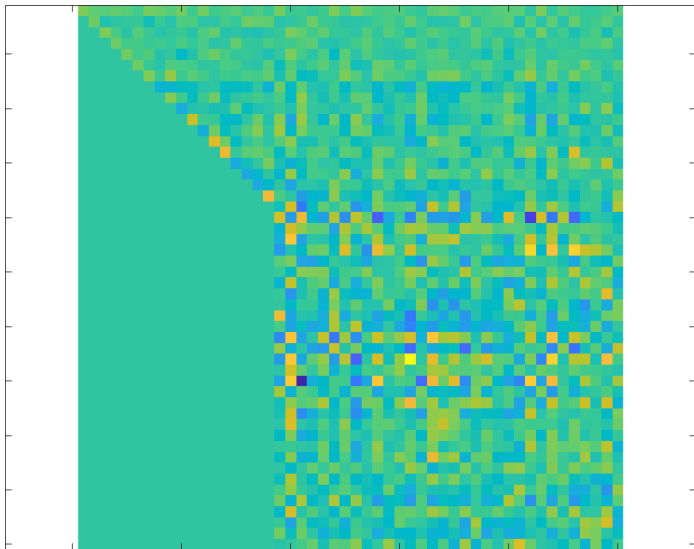
Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



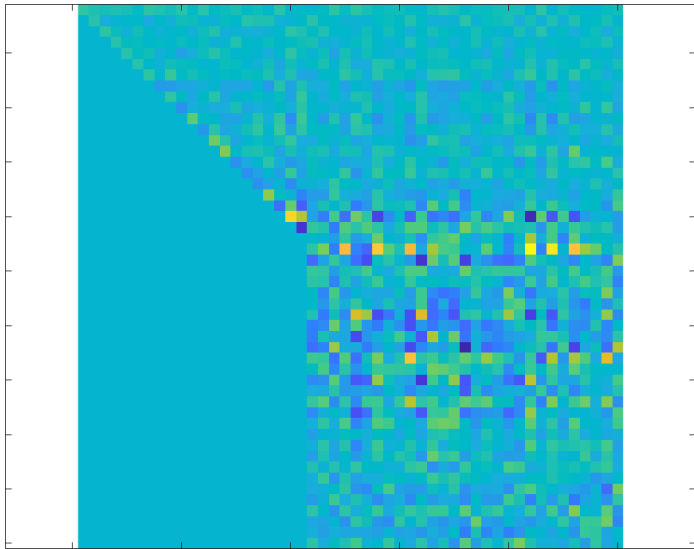
Problem 3(1) Solution



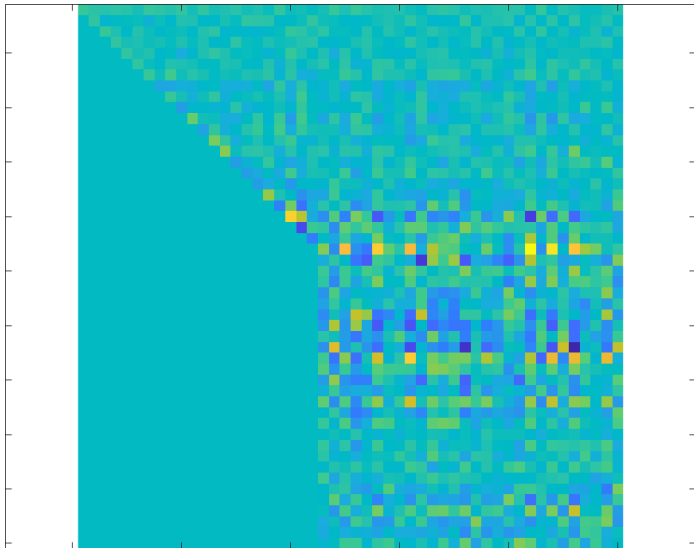
Problem 3(1) Solution



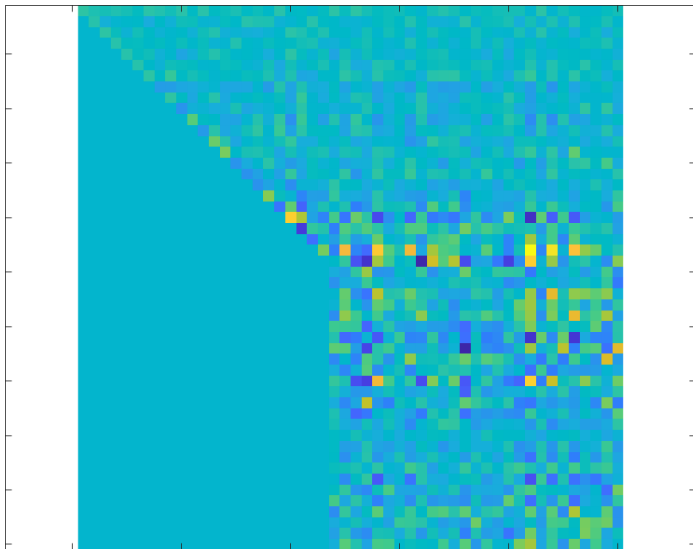
Problem 3(1) Solution



Problem 3(1) Solution



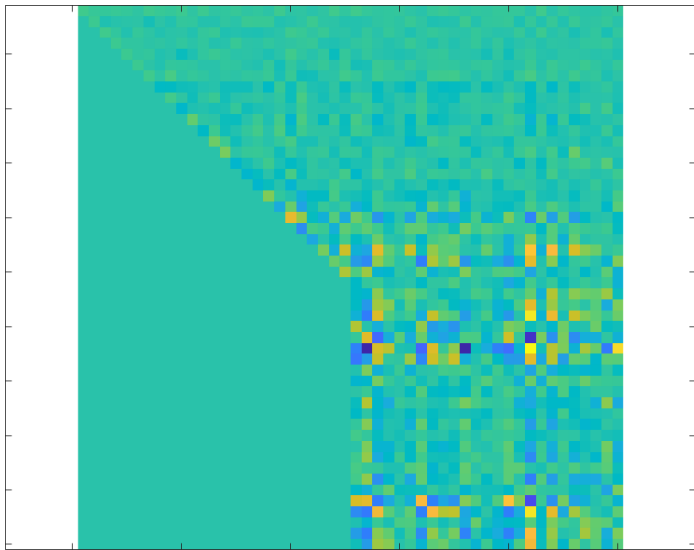
Problem 3(1) Solution



Problem 3(1) Solution



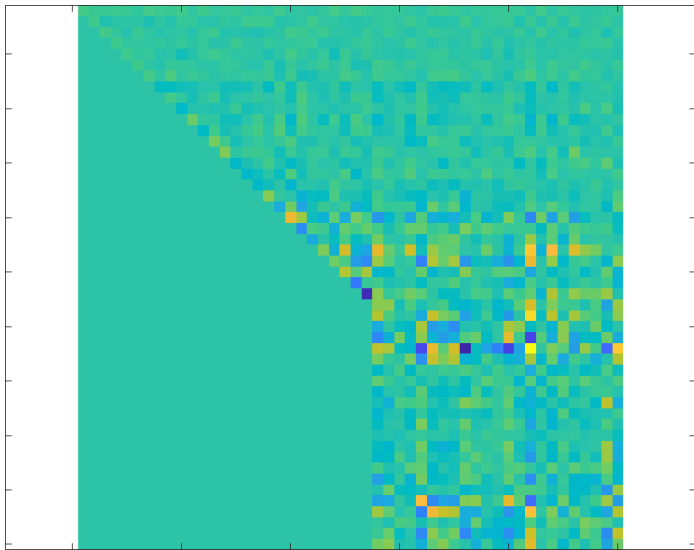
Problem 3(1) Solution



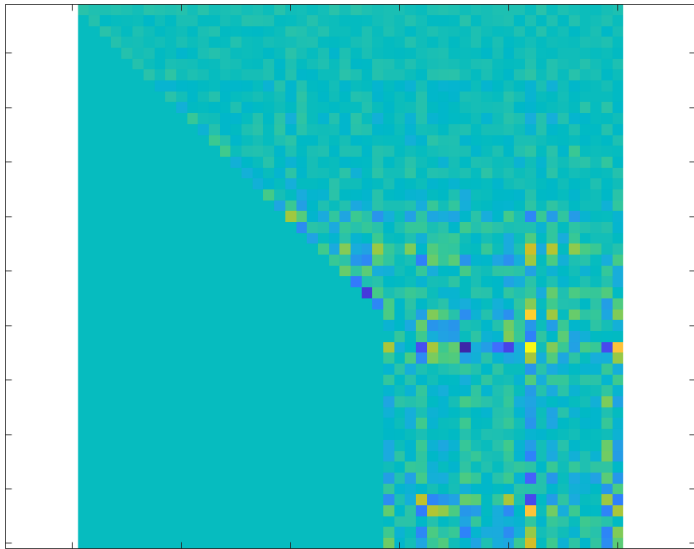
Problem 3(1) Solution



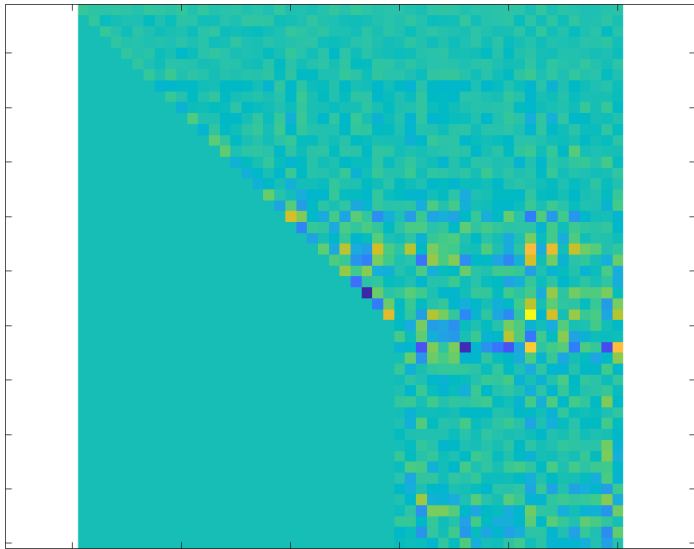
Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



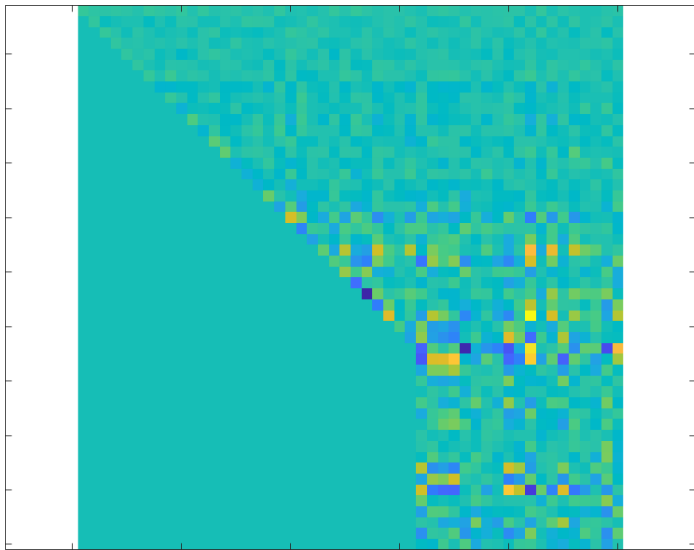
Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(1) Solution



Problem 3(2)

Randomly generate 3 diagonal matrices (30×30), and use Jacobi iteration and Gauss Seidel iteration to solve them respectively. The error of the solution in terms of norm is required to be less than $1/1000$.

```
1 %随机生成三对角矩阵
2 n=30;
3 A=diag(round(rand(1,n)*9)+1,0)+diag(round(rand(1,n-1)*9)+1,1)+diag(round(rand(1,n-1)*9)+1,-1);
4 b=round(rand(n,1)*9)+1;
5 x0=zeros(n,1);
6 %误差
7 eps=1e-3;
8 %最大迭代次数
9 Nmax=500;
```

Problem 3(2) Jacobi Matlab Code

```
1 %Jacobi
2 x = zeros(1,n);
3 y = zeros(1,n);
4 count = 1;
5 while(1)
6     for i = 1:n
7         for j = 1:n
8             if i ~= j
9                 y(i) = y(i) + A(i,j)*x(j);
10            end
11        end
12        y(i)=( b(i) - y(i))/A(i,i);
13    end
14    if max(abs(x-y)) < eps
15        fprintf('迭代结束, 次数%d, 结果: \n',count);
16        disp(y);
17        break;
18    end
19    if count == Nmax
20        fprintf('超过最大迭代次数, 结果: \n');
21        disp(y);
22        break;
23    end
24    count = count + 1;
25    x = y;
26    y(1: n) = 0;
27 end
```

Problem 3(2) Gauss-Deidel Matlab Code

```
1 %Gauss-Seidel
2 x = zeros(1,n);
3 y = zeros(1,n);
4 count = 1;
5 x1 = zeros(1,n);
6 x2 = zeros(1,n);
7 while(1)
8     x1(1:n) = 0;
9     x2(1:n) = 0;
10    for i = 1:n
11        for j = 1: i-1
12            x1(i) = x1(i) + A(i,j) * y(j);
13        end
14        for k = i + 1 : n
15            x2(i) = x2(i) + A(i,k) * x(k);
16        end
17        y(i) = (b(i) - x1(i) - x2(i))/A(i,i);
18    end
19    if max(abs(x-y)) < eps
20        fprintf('迭代结束, 次数%d, 结果: \n', count);
21        disp(y);
22        break;
23    end
24    if count == Nmax
25        fprintf('超过最大迭代次数, 迭代结束, 结果: \n');
26        disp(y);
27        break;
28    end
29    count = count + 1;
30    x = y;
31 end
```

Problem 4

Use appropriate polynomial interpolation to approximate the following ellipse.

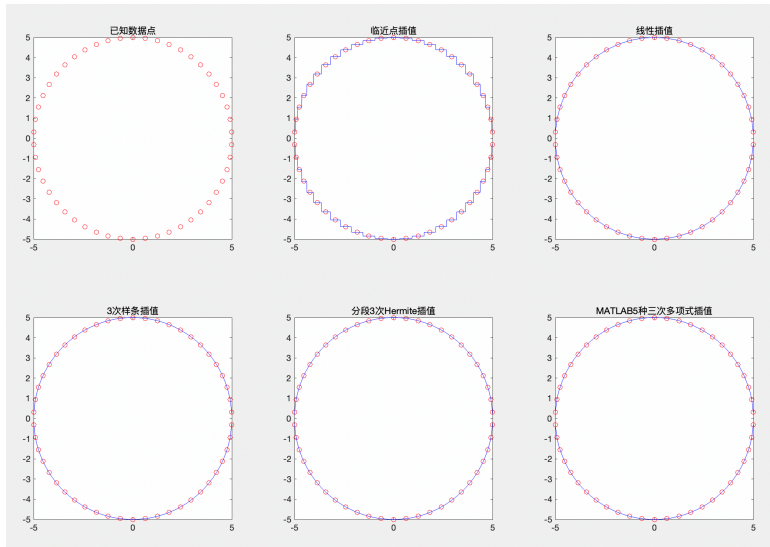
$$x^2 + y^2 = 5^2 \quad (20)$$

$$\rho(\theta) = 2(1 - \sin(\theta)) \quad (21)$$

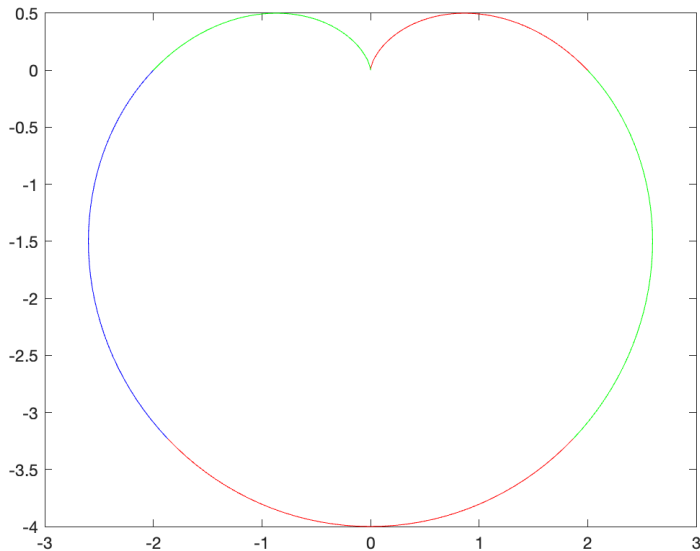
Problem 4(1) Matlab Code

```
1 clc;clear;close all;
2 n = 50;
3 theta = 0:2*pi/n:2*pi;
4 x = 5 * sin(theta);
5 y = 5 * cos(theta);
6 [~, positive] = find(y > 0);
7 [~, negative] = find(y <= 0);
8 X = x(positive);Y = y(positive);
9 X_ = x(negative);Y_ = y(negative);
10 x1 = [-5:0.01:5];
11
12 yi_nearest=interp1(X,Y,x1,'nearest');
13 yi_linear=interp1(X,Y,x1);
14 yi_spine=interp1(X,Y,x1,'pchip');
15 yi_pchip=interp1(X,Y,x1,'pchip');
16 yi_vScubic=interp1(X,Y,x1,'v5cubic');
17 yi_nearest_=interp1(X_,Y_,x1,'nearest');
18 yi_linear_=interp1(X_,Y_,x1);
19 yi_spine_=interp1(X_,Y_,x1,'pchip');
20 yi_pchip_=interp1(X_,Y_,x1,'pchip');
21 yi_vScubic_=interp1(X_,Y_,x1,'v5cubic');
22 figure;
23 axis equal;
24 hold on;
25 subplot(231);
26 plot(X,Y,'ro',X_,Y_,'ro');
27 title('已知数据点');
28 subplot(232);
29 plot(X,Y,'ro',X_,Y_,'ro',xi,yi_nearest,'b-',xi,yi_nearest_,'b-');
30 title('临近点插值');
31 subplot(233);
32 plot(X,Y,'ro',X_,Y_,'ro',xi,yi_linear,'b-',xi,yi_linear_,'b-');
33 title('线性插值');
34 subplot(234);
35 plot(X,Y,'ro',X_,Y_,'ro',xi,yi_spine,'b-',xi,yi_spine_,'b-');
36 title('3次样条插值');
37 subplot(235);
38 plot(X,Y,'ro',X_,Y_,'ro',xi,yi_pchip,'b-',xi,yi_pchip_,'b-');
39 title('分段3次Hermite插值');
40 subplot(236);
41 plot(X,Y,'ro',X_,Y_,'ro',xi,yi_v5cubic,'b-',xi,yi_v5cubic_,'b-');
42 title('MATLAB5种三次多项式插值');
```

Problem 4(1) Solution



Problem 4(1) Solution



Problem 5

我国古代数学著作《九章算数》记载的线性方程组如下：

今有上禾三秉，中禾二秉，下禾一秉，实三十九斗；上禾二秉，中禾三秉，下禾一秉，实三十四斗；

上禾一秉，中禾二秉，下禾三秉，实二十六斗；问上、中、下禾实一秉各几何？

查阅九章算术记载的此问题的解法是什么？指出相当于今天的什么方法。

Problem 5 Solution

$$\begin{array}{ccc} 1 & 2 & 3 \\ 2 & 3 & 2 \\ 3 & 1 & 1 \\ 26 & 34 & 39 \end{array}$$

$$\begin{array}{ccc|ccc|ccc} 1 & 2 & 3 & 3 & 6 & 3 & 0 & 0 & 3 \\ 2 & 3 & 2 & 6 & 9 & 2 & 4 & 5 & 2 \\ 3 & 1 & 1 & \rightarrow 9 & 3 & 1 & \rightarrow 8 & 1 & 1 \\ 26 & 34 & 39 & 78 & 102 & 39 & 39 & 24 & 39 \end{array}$$

$$\begin{array}{ccc|ccc|ccc} 0 & 0 & 3 & 0 & 0 & 3 & 0 & 0 & 3 \\ 4 & 5 & 2 & 20 & 5 & 2 & 0 & 5 & 2 \\ 8 & 1 & 1 & \rightarrow 40 & 1 & 1 & \rightarrow 36 & 1 & 1 \\ 39 & 24 & 39 & 195 & 24 & 39 & 99 & 24 & 39 \end{array}$$

Problem 5 Solution

$$\begin{array}{ccc|ccc}
 0 & 0 & 3 & 0 & 0 & 3 \\
 0 & 5 & 2 & 0 & 5 & 2 \\
 36 & 1 & 1 & \rightarrow & 4 & 1 & 1 \\
 99 & 24 & 39 & 11 & 24 & 39
 \end{array}$$

$$\begin{array}{ccc|ccc|ccc|ccc}
 0 & 0 & 3 & 0 & 0 & 3 & 0 & 0 & 3 & 0 & 0 & 3 \\
 0 & 5 & 2 & 0 & 20 & 2 & 0 & 20 & 2 & 0 & 4 & 2 \\
 4 & 1 & 1 & \rightarrow & 4 & 4 & 1 & \rightarrow & 4 & 0 & 1 & \rightarrow & 4 & 0 & 1 \\
 11 & 24 & 39 & 11 & 96 & 39 & 11 & 85 & 39 & 11 & 17 & 39
 \end{array}$$

Problem 5 Solution

```
1 function split = Split_J(A)
2   [R,~] = size(A);
3   ab_D = A;
4   for i=1:R
5       for j=1:R
6           if(i~=j)
7               ab_D(i,j) = 0;
8           end
9       end
10  end
11  split = abs(eig(ab_D^-1 * (ab_D-A)));
12  split = max(split);
13 end
```

Problem 5 Solution

```
1 function split = Split_G(A)
2     [R,~] = size(A);
3     ab_D = A;
4     ab_L = A;
5     for i=1:R
6         for j=1:R
7             if(i~=j)
8                 ab_D(i,j) = 0;
9             end
10            if(i>j)
11                ab_L(i,j) = -ab_L(i,j);
12            else
13                ab_L(i,j) = 0;
14            end
15        end
16    end
17    ab_U = ab_D - ab_L - A;
18    split = abs(eig((ab_D-ab_L)^-1 * ab_U));
19    split = max(split);
20 end
```

Problem 5 Solution

```
1 if(Split_G(A)<1)
2     i = 1;
3     while(error_G>=Epsilon)
4         X_G(:,i+1) = T_G*X_G(:,i) + C_G;
5         error_G(i) = sqrt(sum((X_G(:,i+1)-X(1,:)).^2,1));
6         i = i+1;
7     end
8     X_G = X_G';
9     error_G
10    X_G
11 else
12     fprintf("Gauss method can't convergent")
13 end
14
15 if(Split_J(A)<1)
16     i = 1;
17     while(error_J>=Epsilon)
18         X_J(:,i+1) = T_J*X_J(:,i) + C_J;
19         error_J(i) = sqrt(sum((X_J(:,i+1)-X(1,:)).^2,1));
20         i = i+1;
21     end
22     X_J = X_J';
23     error_J
24    X_J
25 else
26     fprintf("Jacobi method can't convergent")
27 end
```

Problem 5 Solution

X_G =

0	0	0
13.0000000000000000	2.6666666666666666	2.5555555555555555
10.370370370370370	3.567901234567900	2.831275720164609
9.677640603566530	3.937814357567443	2.815576893766194
9.436264797032974	4.103964504055952	2.785268731618373
9.335600753423241	4.181176587178381	2.767348690739999
9.290099378301079	4.217484184219281	2.758310751086786
9.268906960158219	4.234625109532259	2.753947606925754
9.258934058003243	4.242728092355920	2.751869919094972
9.254224632064396	4.246560272258744	2.750884941139371
9.251998171447713	4.248372905321734	2.750418672636273
9.250945172240087	4.249230327627851	2.750198057501403
9.250447095747631	4.249635917001110	2.750093690083382
9.250211491971466	4.249827775324562	2.750044319126470
9.250100043408136	4.249918531352419	2.750020964629008
9.250047324222052	4.249961462308963	2.750009917053341